A two-level dynamic-weight network model using BGP and hop-by-hop, packet-dependent Dijkstra's algorithm

Justin Polinski
Faculty of ECE
OTH Regensburg
Regensburg, Germany
justin.polinski@t-online.de

Hoang L.M Do
Faculty of ECE
Vietnamese-German University
Ho Chi Minh City, Vietnam
hoangdo191122@gmail.com

***Abstract*—Inter-domain routing on the Internet is organised as a two-level hierarchy: the Border Gateway Protocol (BGP) selects the sequence of Autonomous Systems (ASes) a packet crosses, while a link-state Interior Gateway Protocol such as OSPF—driven by Dijkstra's shortest-path algorithm—forwards the packet inside each AS. In their classical form both layers compute a forwarding path once per routing cycle and commit to it until the next reconvergence. When a link metric changes while a packet is still in flight—because of congestion, shifting load, or a partial failure—the committed path can become suboptimal or even unusable, causing longer routes or lost packets. Recomputing a shortest path continuously as a traveling entity moves, rather than reusing a table computed once, is established practice in road-traffic routing; this paper studies the analogous technique in a computer network, applied specifically beneath a BGP layer that continues to fix the AS-path and border router exactly as in standard operation. The intra-AS route to that fixed point is instead re-solved *at every hop*, using the link weights current at that instant, so each forwarding decision is locally optimal for the present network state without altering the coarser BGP-level decision. On a simulated multi-AS topology subject to mid-transit weight changes, the scheme lowers average path cost and keeps delivery high relative to commit-once routing, at the cost of additional per-hop computation. We report path cost, path stretch, delivery ratio, and computational overhead, and characterise how the benefit grows with the rate of network change.**

***Index Terms*—dynamic routing, Dijkstra's algorithm, OSPF, BGP, shortest path, inter-domain routing, network simulation**

I. INTRODUCTION

The global Internet is not a single network but a collection of tens of thousands of independently administered networks, the Autonomous Systems (ASes). Routing is correspondingly split into two levels. *Inter-AS* routing—deciding which ASes a packet traverses—is handled by the Border Gateway Protocol (BGP) [1]. *Intra-AS* routing—moving a packet across a single AS to the correct exit—is handled by an Interior Gateway Protocol (IGP), most commonly the link-state protocol OSPF, whose forwarding tables are produced by Dijkstra's shortest-path algorithm [2], [3].

Both layers are, in their standard operation, *commit-once* within a routing cycle. Each router computes a path from its current view of the network and installs it in the forwarding table; that table then stays fixed until a topology or metric change triggers a new round of flooding and recomputation. Between two such convergence events the forwarding decision is static. This design is efficient and stable, but it embeds an implicit assumption: that the cost of a link does not change meaningfully while a packet is being delivered along it.

In practice this assumption is often violated. Link metrics track conditions that fluctuate on short timescales—queueing delay under bursty load, traffic engineering adjustments, or the sudden loss of capacity when a link is physically damaged or administratively removed. When such a change occurs *after* a path has been chosen but *before* the packet has finished traversing it, the committed route may no longer be the cheapest available, or may have been broken entirely. The consequences range from unnecessary extra cost and delay to outright packet loss. In latency- or reliability-sensitive settings these outcomes are significant.

This paper studies a routing strategy designed to react to exactly this class of mid-transit change: rather than computing one intra-AS path at the source and committing to it, the shortest-path problem is re-solved at every hop, using the link weights current at that instant. BGP continues to fix the AS-path and the border router exactly as in standard operation; only the intra-AS journey toward that fixed point is re-evaluated continuously. The mechanics of the scheme, and why it reduces to classical Dijkstra whenever no change occurs during transit, are developed in Section IV.

The contributions of this paper are:

- A per-hop dynamic intra-AS routing scheme, operating beneath a BGP-selected AS-path and border router that is fixed for the duration of a packet's transit, re-evaluated continuously as the packet is forwarded.
- A discrete-event simulator of a multi-AS network in which link weights change while packets are in flight, enabling a controlled comparison between commit-once and per-hop routing.
- A quantitative evaluation—path cost, path stretch relative to the offline optimum, packet delivery ratio, and per-hop

computational overhead—showing where the dynamic scheme improves on the static one and at what cost, and how the advantage scales with the rate of change.

The remainder of the paper is organised as follows. Section II positions this work against existing dynamic and adaptive routing techniques. Section III reviews the theoretical foundations: Dijkstra's algorithm, the OSPF link-state protocol, and the BGP path-vector protocol. Section IV presents the principle of dynamic per-hop routing and our concrete approach. Section V reports the experimental comparison against classical static routing, and Section VI concludes.

II. RELATED WORK

Recomputing a shortest path continuously as a traveling entity moves through a network, rather than trusting a table computed once, is an established technique outside computer networking, chiefly in road-traffic routing. Work on emergency-vehicle dispatch recomputes the vehicle's route as congestion changes en route, precisely because a route fixed at departure degrades as traffic conditions evolve during the trip [4]. A parallel line of work casts this as a *dynamic stochastic shortest path problem*: the traveler is allowed to adapt the chosen route while driving, and the routing policy is recomputed at every intersection reached [5]. Efficient data structures for repeatedly updating a shortest-path solution as edge weights change — rather than solving from scratch each time — have also been studied directly under the name *dynamic shortest path problem* [6]. In each case the technique operates on a single, flat graph: there is no notion of first committing to a coarser, higher-level decision and only then navigating dynamically beneath it.

Within computer networking, the closest analogue to reacting quickly to a change is IP Fast-Reroute, which precomputes a backup path so that a router can invoke a local repair immediately upon detecting a failure, without waiting for network-wide reconvergence [7]. This mechanism is *event*-triggered: it activates once, when a failure is detected, and is silent otherwise. It does not recompute on every packet at every hop regardless of whether a change has occurred, and it addresses failures rather than continuous cost fluctuation.

This paper's setting differs from both bodies of work in the same respect: routing here is not a single flat graph. A packet's journey is fixed at a coarse level — which AS-path and which border router to head toward, decided once by BGP, exactly as in standard operation — and only the finer-grained intra-AS path toward that fixed point is subject to continuous, per-hop recomputation. Neither the road-traffic dynamic-routing literature nor IP Fast-Reroute combines a fixed higher-level path decision with a continuously live lower-level one; this paper studies that combination in a BGP/OSPF network and reports the resulting cost, delivery, and computational tradeoffs.

III. THEORETICAL BACKGROUND

This section establishes the three building blocks the method rests on: Dijkstra's shortest-path algorithm as the computational core, OSPF as the intra-AS protocol that runs it, and BGP as the inter-AS protocol that decides the AS-level path. Their interaction defines the two-level routing model used throughout the paper.

A. Dijkstra's Shortest-Path Algorithm

Consider a network modelled as a weighted graph $G = (V, E)$, where the vertices V are routers, the edges E are links, and each edge (u, v) carries a non-negative cost $w(u, v)$. Given a source s, Dijkstra's algorithm [2] computes the minimum-cost path from s to every other vertex.

The algorithm maintains a tentative distance $d(v)$ for each vertex, initialised to $d(s) = 0$ and $d(v) = \infty$ otherwise. It repeatedly selects the unvisited vertex u of smallest $d(u)$, marks it final, and *relaxes* each outgoing edge:

$$d(v) \leftarrow \min\big(d(v),\ d(u) + w(u, v)\big). \tag{1}$$

Because all weights are non-negative, once a vertex is selected its distance is provably optimal and never revised. The process yields a shortest-path tree rooted at s. With a binary-heap priority queue the running time is $\mathcal{O}\big((|V| + |E|) \log |V|\big)$.

The property that matters for this work is that Dijkstra solves the problem for *one fixed snapshot* of the weights w. If a weight changes, the previously computed tree is no longer guaranteed optimal and must be recomputed. Classical routing performs this recomputation only at discrete convergence events; the method proposed here performs it at every hop.

B. OSPF: Link-State Intra-AS Routing

Open Shortest Path First (OSPF) [3] is the dominant link-state IGP and provides the intra-AS routing in our model. Every router floods descriptions of its own links—Link-State Advertisements (LSAs)—to all other routers in the area. From the received LSAs each router assembles an identical copy of the Link-State Database (LSDB), a complete map of the intra-AS topology and its link costs, where cost is typically derived from link bandwidth.

Holding this full map, each router independently runs the Shortest Path First computation—Dijkstra's algorithm of Section II-A—rooted at itself, and installs the resulting next hops into its forwarding table. Two facts follow. First, forwarding is inherently hop-by-hop: each router applies its own table and never dictates the full downstream path. Second, and central to this paper, the table is recomputed only when the LSDB changes. A metric change must be detected, re-flooded, and reprocessed before any router acts on it; until this *reconvergence* completes, every router forwards according to the previous, now-stale, shortest-path tree. It is precisely this lag—static forwarding between convergence events—that allows a packet to follow a suboptimal or broken path when conditions change during its transit.

C. BGP: Path-Vector Inter-AS Routing

Where OSPF governs a single AS, the Border Gateway Protocol (BGP) [1] governs routing *between* ASes and is the protocol that holds the Internet together. BGP is a *path-vector* protocol: a router advertises reachable IP prefixes together with the `AS_PATH` attribute, the ordered list of ASes a route

traverses. Carrying the full AS list serves two purposes—loop detection (a router rejects any route whose `AS_PATH` already contains its own AS) and a coarse path-length metric.

Each BGP speaker stores routes in three tables: the *Adj-RIB-In* (all routes learned from neighbours), the *Loc-RIB* (the single best route chosen for each prefix), and the *Adj-RIB-Out* (routes re-advertised onward). When several routes to the same prefix exist, the best-path selection process compares them in a fixed order of attributes: higher `LOCAL_PREF` first (local exit policy), then shorter `AS_PATH`, then lower `ORIGIN` type, then lower `MED` (preferred entry point advertised by a neighbour), and finally the lowest IGP cost to the route's next hop. That last tie-breaker is the seam between the two layers: BGP defers to the intra-AS shortest-path distance—the OSPF/Dijkstra cost—to choose among otherwise equal exits.

The two protocols therefore compose into a two-level routing decision. BGP fixes the *AS-level* path and, with it, the border router at which the packet must leave the current AS. Dijkstra-based intra-AS routing then delivers the packet across the AS to that border router (or, in the destination AS, directly to the target router). In this paper, the BGP decision is made once, exactly as in standard operation, and held fixed for the packet's transit; it is the intra-AS Dijkstra computation of Section II-A that is instead re-evaluated at every hop against the current link weights. Section IV builds on this composition: the destination BGP selects is fixed, the route taken to reach it is not.

IV. METHODOLOGY

A. Problem of Dynamic Changes

Consider the intra-AS routing problem introduced in Section III. Under standard OSPF operation, if the topology changes during a routing cycle—for instance a link weight changes, or a link fails outright—the forwarding tables in use are not updated until the next reconvergence. Such changes commonly arise from fluctuating traffic load on a link, or from a link being cut through physical damage or administrative action. A packet already in flight when the change occurs continues to be forwarded according to the now-stale table, which can result in a non-optimal route for that packet, or in outright packet loss if the change breaks a link the stale path depends on. Depending on the application, the resulting extra cost, delay, or loss can be significant. This is the gap the dynamic scheme below is designed to close, without altering the coarser, BGP-level decision of which AS-path and border router to head toward.

B. Three Routing Strategies Compared

To isolate the effect of per-hop recomputation, this paper compares three strategies for the intra-AS portion of a packet's journey. All three share the same BGP layer: the AS-path and target border router (or, inside the destination AS, the target router itself) are chosen once, using the best-path selection process of Section III, and are held fixed for the remainder of the packet's transit in every strategy. The strategies differ only in how the intra-AS route to that fixed point is computed.

- **Optimal.** A single Dijkstra computation on the graph's *final* edge weights—i.e., the weights as they stand after all changes for the run have occurred. This strategy is not something either of the other two could realistically achieve, since it presupposes knowledge of weight changes before they happen. It serves purely as a ground-truth lower bound on path cost.
- **Static.** The classical commit-once behaviour of OSPF. A single Dijkstra computation is performed once, before the packet begins its journey, using the weights in effect at that moment. The resulting path is fixed and is not revisited even if weights change while the packet is still traversing it. For comparison purposes the completed static path is costed using the network's final weights, i.e., the cost the packet would actually have incurred had it blindly followed the original plan to the end.
- **Dynamic (proposed).** The per-hop scheme detailed in Section IV-C: the intra-AS shortest path to the fixed BGP-selected destination is recomputed at every hop, using whatever weights are current at that instant.

When no weight changes occur during a packet's transit, Static and Dynamic compute identical paths at identical cost, and both match Optimal; the three strategies only diverge once weights change mid-transit, which is exactly the condition the experiments in Section V vary.

C. The Per-Hop Dynamic Routing Algorithm

The core mechanism is a hop-by-hop routing decision. Rather than computing one path at the source router and holding it fixed, every router the packet currently occupies re-solves the intra-AS shortest-path problem to the BGP-fixed destination, using the link weights as they stand at that instant, and forwards to the first router on the resulting path. This is repeated at every subsequent router the packet reaches. Algorithm 1 summarises the procedure for a single packet.

Two properties follow directly from this construction. First, if the weights of G never change during the packet's transit, the shortest path recomputed at every hop is simply the suffix of the path that classical, one-shot Dijkstra would have produced at the source—so Dynamic reduces exactly to Static (and to Optimal) whenever no change occurs, and nothing is lost by recomputing more often than necessary. Second, whenever a weight change does occur, the very next recomputation is already aware of it, so the packet's remaining route is locally optimal for the network state at that moment, rather than for the state that held when the packet departed. The price paid for this adaptivity is additional computation: whereas Static performs one Dijkstra run per packet, Dynamic performs one per hop (Section IV-G).

D. Weight-Change (Churn) Model

To subject both strategies to a controlled, repeatable rate of mid-transit change, weight changes are generated by a cascading probabilistic process evaluated once per hop the packet takes. At each hop, one link in the network is changed with probability p_1; conditional on that change occurring, a

Algorithm 1 Per-hop dynamic intra-AS routing for one packet

Require: source router s, BGP-fixed destination t (border router or, in the destination AS, the target router), live weighted graph G
Ensure: the sequence of routers the packet traverses

```
current ← s
path ← [s]
while current ≠ t do
    weights of G may change here (Section IV-D)
    P ← Dijkstra shortest path from current to t on the current G   ▷ recomputed fresh, live weights
    if P does not exist then
        fail (no route available)
    end if
    next ← second vertex of P
    forward packet to next
    append next to path
    current ← next
end while
return path
```

second, distinct link is also changed with probability p_2; a third with probability p_3; and a fourth with probability p_4. The roll stops at the first failed draw. The changed link is drawn uniformly among links not yet locked (Section IV-D), assigned a new weight drawn uniformly at random from a fixed range, and the network's routing state is reconverged before the packet's next hop is decided. Unless otherwise noted, the base probabilities used are $(p_1, p_2, p_3, p_4) = (1.0, 0.6, 0.2, 0.1)$; the sweep in Section V.C scales all four by a common *intensity multiplier* to study how the benefit of dynamic routing depends on the rate of change, as promised in the abstract.

Once the packet has actually traversed a given link during the current run, that link's weight is treated as fixed ("locked") for the remainder of the run and excluded from further draws, since a link already crossed cannot retroactively affect the packet's cost. This also lets the simulator log, for every change, whether it landed on a link still ahead of the packet on its originally planned route—distinguishing changes that could plausibly influence the outcome from ones the packet was never going to use anyway.

E. Failure Model

A second, independent set of experiments (Section V.E) studies robustness to outright link failure rather than cost fluctuation. Each attempt to cross a link is dropped independently with a fixed per-hop failure probability p_{fail}. Under Static routing, any dropped hop along the fixed path fails the delivery outright. Under Dynamic routing, a dropped attempt is treated as evidence that the attempted link is temporarily unusable: the edge is removed from the working topology, the network's routing state is reconverged, and a new next hop is computed from the router's current position—repeated up to a fixed number of retries per hop before the delivery is counted as failed. The edge is restored once the packet has moved past that hop, so the failure is transient rather than a permanent topology change.

F. Simulation Environment

Experiments are run on a discrete-event simulator implementing the model of Sections III–IV-E directly, so the measurements in Section V come from the same code path used by the accompanying live demonstrator, rather than a separate re-implementation. A topology is generated by first fixing a number of ASes and a number of routers per AS; within each AS, routers are connected into a base ring plus a small number of additional random intra-AS links, each assigned a random integer weight; ASes are then connected to their neighbours in an AS-level ring via one randomly chosen border-router pair per adjacent AS pair, with inter-AS link weights drawn from a higher range than intra-AS links to reflect their typically higher cost. Each AS originates exactly one IP prefix. BGP best-path selection (Section III.C) is run over this topology to populate every router's Loc-RIB before a run begins, and is re-run whenever a weight change requires reconvergence, as described above. We note this AS-level topology is deliberately simple (a ring rather than a richer random inter-AS graph); Section V.E returns to this point when discussing how delivery ratio scales with topology size.

G. Evaluation Metrics

The following quantities are reported in Section V, computed per simulated packet (a random source/destination router pair) unless stated otherwise:

- **Path cost** — the sum of edge weights along a completed path, evaluated separately for the Static, Dynamic, and Optimal strategies of Section IV-B.
- **Path stretch** — a strategy's path cost divided by the Optimal cost for the same source/destination pair; a value of 1.0 indicates the strategy matched the ground-truth lower bound.
- **Cost saving** — the percentage reduction in path cost of Dynamic relative to Static, $(\text{cost}_{\text{static}} - \text{cost}_{\text{dynamic}})/\text{cost}_{\text{static}}$, reported both as a per-trial distribution and as a function of the churn intensity multiplier of Section IV-D.
- **Delivery ratio** — the percentage of trials in which a strategy successfully reaches the destination under the failure model of Section IV-E, as a function of p_{fail}.
- **Computational overhead** — the number of Dijkstra recomputations performed per packet (one for Static, one per hop for Dynamic) and the wall-clock runtime per trial, both reported in absolute terms and as a function of network size.

V. EXPERIMENTAL RESULTS / PERFORMANCE EVALUATION

In order to validate the proposed approach, we conducted a series of simulations using different network topologies. The simulations were only done in a virtual environment, which is not a real world scenario. Therefore the results are only a

first indication of the possible performance of the proposed approach, not a final evaluation.

A. Experimental Setup

Unless noted otherwise, results below use a topology of 6 ASes with 10 routers per AS, generated as described in Section IV-F, with the base churn intensity of Section IV-D. Each reported figure is averaged over the stated number of independent trials, where a trial is one randomly chosen source/destination router pair simulated from a freshly generated topology. The three strategies compared — *Optimal*, *Static*, and *Dynamic* — are defined in Section IV-B.

B. Path Cost: Dynamic vs. Static

Across 295 (out of 300) successful trials, the statically committed path had an average cost of 44.19, compared to 41.84 for the dynamically rerouted path and 40.85 for the policy-free optimal reference. This corresponds to an average cost saving of 3.51% for dynamic routing. Notably, the distribution is heavily right-skewed: the median saving across all trials is 0%, since most weight changes occur on links that are not part of the traveled path — the benefit of dynamic rerouting is event-driven rather than uniform, with individual trials saving up to 81.5%. The five unsuccessful trials correspond to cases where no route existed between the sampled source and destination after weight changes; these are excluded from the averages above and are consistent with the delivery-ratio behaviour studied separately in Section V-E.

Table I. Comparison of routing costs

Static Cost	Dynamic Cost	Optimal Cost
44.19	41.84	40.85
26.22	23.56	22.03
1.00	1.00	1.00
25.00	24.00	24.00
41.00	40.00	40.00
59.00	57.00	57.00
135.00	108.00	94.00

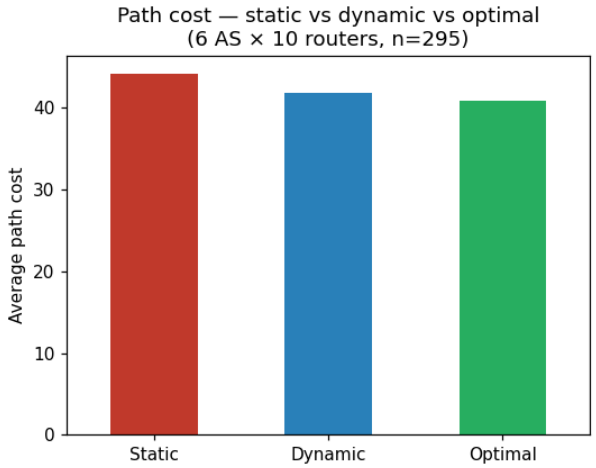

Figure 1. Average path cost — Static, Dynamic, and Optimal — over 300 trials on a 6 AS × 10 routers topology.

C. Path Stretch Relative to Optimal

To express the same result independent of the absolute cost scale of a given topology, Fig. 2 reports path stretch (Section IV-G) for Static and Dynamic over the same trials as Section V-B. A stretch of 1.0 (dashed line) corresponds to matching the Optimal reference exactly.

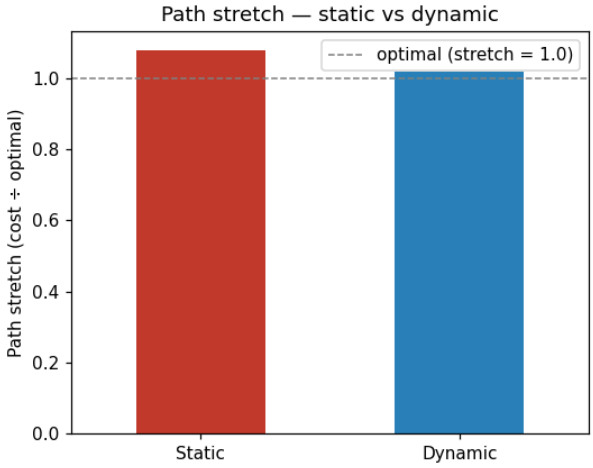

Figure 2. Average path stretch (cost ÷ optimal cost) for Static and Dynamic routing, with the optimal reference line at 1.0.

D. Cost Savings vs. Churn Intensity

Section V-B reports a single churn intensity. To characterise how the benefit of dynamic routing grows with the rate of network change, we sweep the intensity multiplier of Section IV-D from 0× (no mid-transit changes at all) up to 10× the base rate, running 150 trials at each level. Fig. 3 shows the resulting average cost saving. As expected, the saving is 0% at zero churn (Static and Dynamic coincide when nothing changes, as noted in Section IV-C) and increases with the rate of change, since a higher churn rate both increases the chance a change lands on the packet's remaining route and gives Dynamic more opportunities to react to it.

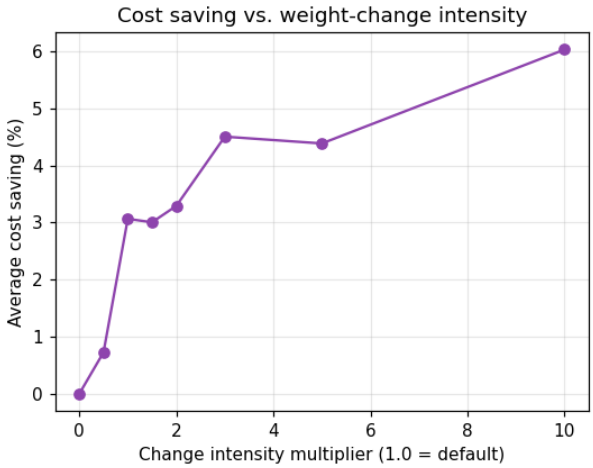

Figure 3. Average cost saving of Dynamic over Static as a function of the churn intensity multiplier (150 trials per level).

E. Delivery Ratio Under Link Failures

Beyond cost, Dynamic routing's ability to route around a dropped link (Section IV-E) should also improve packet delivery ratio relative to Static, which has no mechanism to react to a failed hop. Fig. 4 sweeps the per-hop failure probability p_{fail} from 0 to 0.5 over 150 trials per level, on the base 6 AS $\times$ 10 routers topology. Dynamic maintains a visibly higher delivery ratio than Static as p_{fail} increases, consistent with its ability to exclude a failed edge and reconverge before retrying.

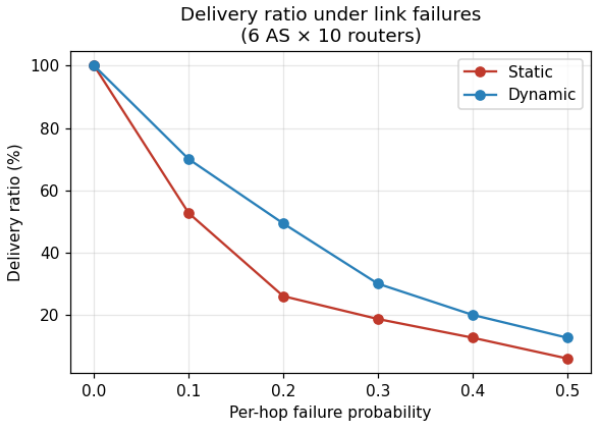

Figure 4. Delivery ratio for Static and Dynamic routing as a function of the per-hop failure probability p_{fail}.

We additionally swept the number of ASes (5–10, routers per AS fixed at 10) and, separately, the number of routers per AS (5–20, AS count fixed at 6), repeating the p_{fail} sweep at each size. Increasing the number of ASes alone produced little change in the Static/Dynamic delivery gap, because in our topology model (Section IV-F) ASes are connected in a ring, so each AS has only two inter-AS neighbours regardless of how many ASes exist, limiting Dynamic's opportunities to reroute at the inter-AS level. Increasing the number of routers per AS, by contrast, gives Dynamic meaningfully more intra-AS alternative paths to route around a failure, and the delivery-ratio gap between Static and Dynamic widens accordingly. This is a useful negative result in its own right: it indicates that dynamic routing's benefit under failure is bounded by the richness of alternative paths in the topology, not by the algorithm itself.

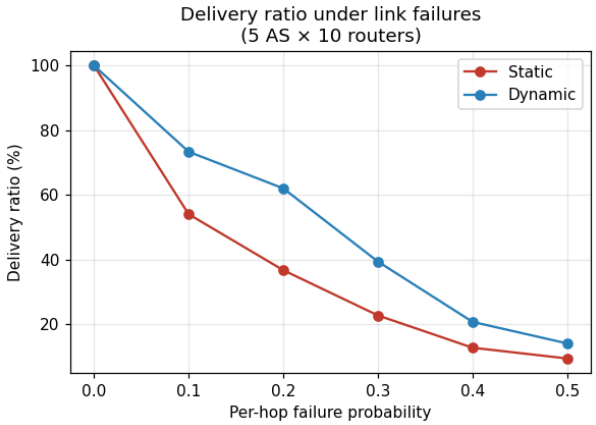

Figure 5. Delivery ratio vs. p_{fail} across AS counts 5–10 (routers per AS fixed at 10). The Static/Dynamic gap is largely unaffected, since the AS-level topology is a ring.

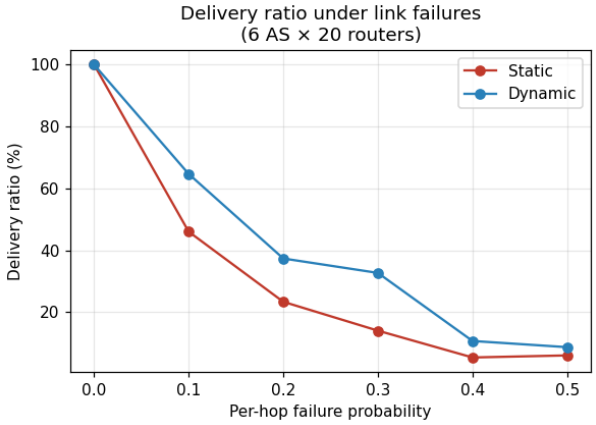

Figure 6. Delivery ratio vs. p_{fail} across routers-per-AS counts 5–20 (AS count fixed at 6). More intra-AS alternative paths widen the Static/Dynamic gap.

F. Computational Overhead

The adaptivity of Dynamic routing is not free: Static performs one Dijkstra computation per packet, while Dynamic performs one per hop plus a network-wide reconvergence whenever a weight change is drawn (Section IV-D). Fig. 7 reports the average number of Dijkstra recomputations per packet for both strategies on the base topology (log scale, since the gap spans more than an order of magnitude), together with the average wall-clock runtime per trial.

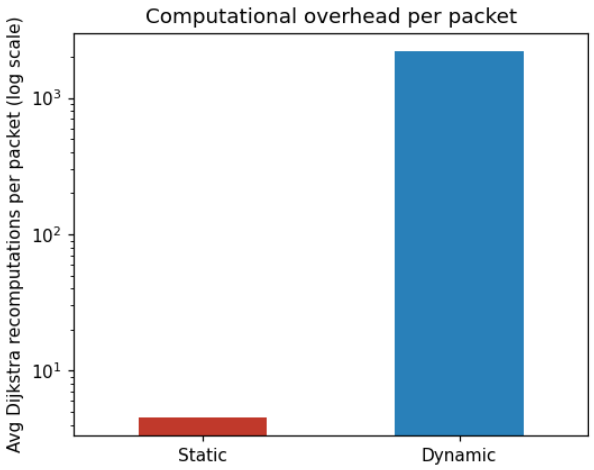

Figure 7. Average Dijkstra recomputations per packet, Static vs. Dynamic (log scale).

To characterise how this overhead scales, we swept network size from 3 ASes × 5 routers up to 8 ASes × 10 routers (40 trials per size) and recorded both the average per-trial runtime and the average number of Dynamic Dijkstra recomputations per packet. Fig. 8 shows both curves. Runtime grows with network size roughly as expected from the $\mathcal{O}((|V|+|E|)\log|V|)$ cost of a single Dijkstra run (Section III.A) multiplied by the number of hops and reconvergences a packet triggers; the number of recomputations per packet grows more gently, since it is driven primarily by path length and churn rate rather than by graph size directly.

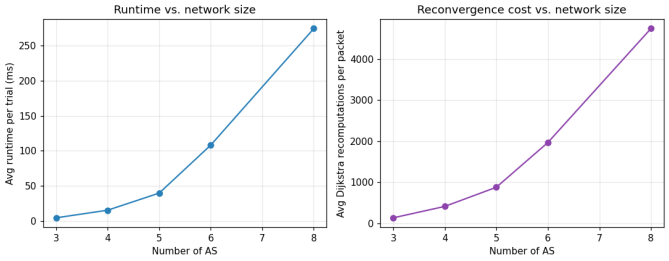

Figure 8. Left: average per-trial runtime vs. number of ASes. Right: average Dynamic Dijkstra recomputations per packet vs. number of ASes (40 trials per size).

Overall, the results in this section indicate that per-hop dynamic routing reduces average path cost and improves delivery ratio relative to commit-once static routing, with the size of both benefits scaling with the rate of network change (Section V-D) and, for delivery, with the richness of alternative intra-AS paths (Section V-E) — at a measurable but bounded computational cost (this section).

VI. CONCLUSION

This paper presented a two-level routing scheme that keeps BGP's inter-AS path selection unchanged while replacing OSPF's commit-once intra-AS forwarding with per-hop Dijkstra recomputation. The method requires no change to the BGP layer: the AS-path and border router stay fixed, and only the finer route toward that fixed point adapts to current link weights at every hop. Simulation on a 6-AS, 10-router-per-AS topology showed Dynamic routing cutting average path cost by 3.51% over Static, with individual trials saving up to 81.5% when weight changes landed on the packet's remaining route. The saving grows with churn rate, from 0% at zero churn to roughly 6% at 10x the base intensity. Under link failure, Dynamic sustained a consistently higher delivery ratio than Static, an advantage that widens with more intra-AS alternative paths but stays flat when only AS count increases, since the ring-shaped AS-level topology limits inter-AS rerouting options. This gain comes at a cost: Dynamic runs one Dijkstra computation per hop instead of one per packet, an overhead spanning more than an order of magnitude on the tested topologies.

These results indicate per-hop dynamic routing is worth deploying where link conditions change often and topology offers route diversity, and offers little benefit otherwise. The findings come from a discrete-event simulator with a simplified ring-shaped inter-AS topology, not a deployed network, so they should be read as an initial indication rather than a final verdict. Testing on denser, more realistic inter-AS graphs, and measuring overhead against production-scale router hardware, are the natural next steps.

REFERENCES

[1] Y. Rekhter, T. Li, and S. Hares, "A border gateway protocol 4 (BGP-4)," Internet Engineering Task Force (IETF), Request for Comments RFC 4271, 2006.
[2] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.
[3] J. Moy, "OSPF version 2," Internet Engineering Task Force (IETF), Request for Comments RFC 2328, 1998.
[4] H. Su, Y. D. Zhong, B. Dey, and A. Chakraborty, "EMVLight: A decentralized reinforcement learning framework for efficient passage of emergency vehicles," *arXiv preprint arXiv:2109.05429*, 2021.
[5] N. Levering, M. Boon, M. Mandjes, and R. Núñez-Queija, "A framework for efficient dynamic routing under stochastically varying conditions," *arXiv preprint arXiv:2111.11095*, 2021.
[6] Sunita and D. Garg, "Dynamizing dijkstra: A solution to dynamic shortest path problem through retroactive priority queue," *Journal of King Saud University – Computer and Information Sciences*, vol. 33, no. 3, pp. 364–373, 2021.
[7] M. Shand and S. Bryant, "IP fast reroute framework," Internet Engineering Task Force (IETF), Request for Comments RFC 5714, 2010.